

Safety Helmet Detection System Using YOLOv8

1. Project Overview

This project implements a complete Computer Vision object-detection pipeline using YOLOv8 to identify whether people in an image are wearing a safety helmet. Not wearing a helmet on construction sites, in factories, or while riding a motorcycle is a common safety risk, and manually monitoring this is slow and error-prone. The system detects three classes (Person, Helmet, and Without Helmet) and draws bounding boxes with confidence scores on new, unseen images. The project covers the full workflow: problem definition, data collection, manual annotation, dataset preparation, model training, evaluation, inference, and error analysis.

2. Tools and Technologies Used

Python, Google Colab, Google Drive, YOLOv8 (Ultralytics), Roboflow (data annotation and dataset management), OpenCV, and Matplotlib.

3. Dataset Preparation

The base dataset was sourced from Roboflow Universe (helmet-detection-kkt4z), which originally contained helmet and without_helmet annotations. After downloading, the dataset was manually reviewed and cleaned, corrupt and unclear images (blurry, static/noise-affected, or unrecognizable) were removed, leaving 211 usable images. A third class, person, was added and manually annotated across 158 images using Roboflow's annotation tool, satisfying the task's minimum manual-annotation requirement of 100 images. The final dataset was split into training, validation, and test sets and exported in YOLOv8 format.

Split	Images	Percentage
Train	148	70%
Validation	45	21%
Test	18	9%

Preprocessing applied: Auto-orientation and resizing (fit within 640x640 with black-edge padding), matching YOLOv8's standard input size.

4. Dataset Configuration (data.yaml)

path: root folder of the dataset. train / val / test: relative paths to each image split. names: the class-index-to-label mapping (0: person, 1: helmet, 2: without_helmet). This file tells the YOLOv8 training script exactly where to find the images and how to interpret the label files.

5. Model Training

A YOLOv8n (nano) model was trained using pretrained COCO weights as the starting point, on Google Collab with a Tesla T4 GPU.

Parameter	Value
Model	YOLOv8n
Epochs	30
Image size	640 x 640
Batch size	8
Platform	Google Colab (Tesla T4 GPU)
Training time	~2.3 minutes

The best-performing checkpoint (best.pt) was automatically saved by Ultralytics based on validation performance.

6. Evaluation Results

Metric	Score
Precision	0.935
Recall	0.940
mAP@0.5	0.963
mAP@0.5:0.95	0.766

Per-Class Performance

Class	Precision	Recall	mAP@0.5
Person	1.000	0.965	0.994
Helmet	0.890	0.909	0.942
Without Helmet	0.916	0.945	0.954

What the Metrics Mean

Precision measures how many of the model's positive detections were actually correct — a high precision means few false alarms. Recall measures how many of the actual objects in the images the model successfully found — a high recall means few objects were missed. mAP@0.5 (mean Average Precision at IoU 0.5) summarizes precision and recall together at a relatively lenient overlap threshold, while mAP@0.5:0.95 averages this across stricter overlap thresholds, making it a tougher, more complete measure of localization quality.

Best-performing class: Person, with mAP@0.5 of 0.994 — person-shaped regions are large and visually consistent across images, making them easier for the model to localize. Weakest-performing class: Helmet, with mAP@0.5 of 0.942 — helmets are smaller objects and can visually blend into hair, shadows, or cluttered backgrounds, making them harder to detect precisely.

7. Inference Results

The trained model (best.pt) was run on 18 unseen test images at a confidence threshold of 0.35. All 18 images produced predictions with visible bounding boxes, class labels, and confidence scores, exceeding the task's minimum requirement of 15 prediction images. In most cases the model correctly identified person, helmet, and without_helmet regions; a smaller number of images showed the 'person' box being missed while the helmet/without_helmet box was still detected correctly (see Error Analysis).

8. Error Analysis

At least 10 weak or incorrect predictions were manually reviewed by comparing model output against the test images and the validation confusion matrix.

Image	Actual	Prediction	Error Type	Possible Reason
-25_png.jpg	Person+Helmet	Only Helmet	False Negative	Person partially occluded
-80_png.jpg	2 Persons+2 Helmets	2 Persons, 1 Helmet	False Negative	Overlapping helmets
img129.jpg	Person+no_helmet	Only no_helmet	False Negative	Person cropped/small
img212.jpg	Person+no_helmet	Only no_helmet	False Negative	Low lighting/angle
img263.jpg	Person+no_helmet	Only no_helmet	False Negative	Person out of frame
img276.jpg	Person+no_helmet	Only no_helmet	False Negative	Small person size
img38.jpg	Person+no_helmet	Only no_helmet	False Negative	Occlusion/low contrast
Conf. Matrix	Helmet	Background	False Negative	Blended w/ background

Conf. Matrix	Without helmet	Background	False Negative	Weak head/hair contrast
Conf. Matrix	Person (x6)	Background	False Negative	Partial visibility

Suggested Improvements

- Add more training images for the 'person' class, especially with partial/occluded bodies.
- Increase training epochs (50-60) since loss curves were still gently decreasing at epoch 30.
- Use a larger base model (YOLOv8s) for better small-object detection capacity.
- Balance the dataset more evenly across all three classes.
- Add more low-light and occluded examples to improve robustness.

9. What I Learned

- How to prepare a Computer Vision dataset from a public source and clean it for real training use.
- How to perform manual bounding-box annotation and understand the YOLO label format.
- How to configure and train a YOLOv8 model end-to-end on Google Colab.
- How to evaluate an object-detection model using precision, recall, and mAP metrics.
- How to interpret a confusion matrix and loss curves to diagnose model behavior.
- How to perform structured error analysis and translate it into concrete improvement ideas.
- The importance of data quality and diversity in determining final model performance.

10. Future Improvements

- Add more images and improve class balance for a more robust dataset.
- Add more difficult (occluded, low-light) examples.
- Train a larger YOLO model (YOLOv8s/m) for higher accuracy.
- Improve annotation quality with a second review pass.
- Deploy the model as a web app (Streamlit/Gradio) for live demonstration.
- Test on real-time video or CCTV-style footage.

11. Conclusion

This project successfully demonstrates a complete, portfolio-ready Computer Vision pipeline for safety helmet detection using YOLOv8 from problem definition and data collection through manual annotation, training, evaluation, inference, and error analysis. The final model achieved strong results ($mAP@0.5 = 0.963$), and the identified error patterns provide a clear, actionable roadmap for future improvement.